Exact PTO Equivalence Binary Multiplication

Metric-Extracted Controller

`analyses/metric_extraction/DFST/mult2/warmup_25000_steps_500000/`

1 Statement and Scope

Controllers. The exported table `minimized_transition_table.pdf` defines controller C, with initial state 0 and halt state 60. The function `mult_grid(2)` defines expert $\mathcal{E}$, with initial state 0 and halt state 31. The adjacent expert export matches this function entry for entry. All tables retain the exported state numbers.

Input Domain. Let $D = \{0, 1\}$. Row 0 initially contains the contiguous word $a\,x\,b$, starting at column 0, with $a \in D^m$, $b \in D^n$, and $m, n \geq 1$. All remaining cells contain blanks. The head starts at $(0, 0)$. Write $A = \mathrm{val}_2(a)$ and $B = \mathrm{val}_2(b)$. The domain permits arbitrary operand lengths, unequal lengths, and leading zeros.

Execution Convention. The grid uses $\mathbb{Z}^2$, with coordinates increasing rightward and downward. The symbol λ denotes a blank. A transition (q', w, M) writes w, performs motion M, and enters state q'. Motions L, R, D, U move one cell. Motion S writes and stops without moving.

Policy-Trajectory Observations. For input $a\,x\,b$, define the policy-trajectory observation (PTO) by

$$\mathrm{PTO}(a\,x\,b) = \big((r_t, w_t, M_t)\big)_{t=0}^{T-1},$$

where r_t, w_t, and M_t denote the read symbol, written symbol, and motion at controller step t. A finite execution contributes the final S action exactly once, without halt-loop padding.

Theorem 1 (Exact PTO Equivalence)**.** *For every input in the stated domain,* $\mathrm{PTO}_C(a\,x\,b) = \mathrm{PTO}_{\mathcal{E}}(a\,x\,b)$. *Both executions terminate on the same step. The complete grids and head positions agree at every step, including the final configuration. The final grid contains only the* $(m + n)$*-bit representation of* AB*, with leading-zero padding, on row* $n - 1$ *in columns* $-n, \ldots, m - 1$. *The final head stops at* $(-n - 1, n - 1)$.

The theorem concerns autonomous executions of the exported discrete controller on the stated input domain.

2 Expert Behavior

2.1 Copying and Partial Products

Write $b = b_{n-1} \cdots b_0$, so $B = \sum_{i=0}^{n-1} b_i 2^i$. Assign global weight 2^{m-1-z} to column z. Column $m-1$ carries the units digit of the final product.

Lemma 1 (Copy Staircase)**.** *Before product formation, the expert constructs exactly n occupied rows. For $0 \le i < n$, row i contains*

$$a\, x\, b_{n-1} \cdots b_i \quad \text{starting in column } -i. \tag{1}$$

The last row contains one right-operand bit. All remaining cells contain blanks.

Proof. Row 0 contains the input. On row i, the expert scans to the first blank, steps left to b_i, then steps left again. A digit at this position implies at least two remaining right-operand bits. The expert copies every symbol except b_i one row down and one column left, proceeding from right to left. Every destination contains a blank before writing; the source word remains unchanged.

The first descent reaches a blank on the new row. Each later descent reaches the copied right neighbor: a digit, except directly below the last digit of a, where the copied separator x occupies the lower cell. The cell below the source separator contains a digit. After copying the first digit of a, the expert reads the blank immediately to the left and descends to the first digit of the new row.

The new row satisfies (1) for $i+1$, with one fewer right-operand bit. When one right-operand bit remains, the second left step reads x and starts product formation. Thus $n-1$ copies produce exactly n occupied rows. □

Lemma 2 (Aligned Partial Products)**.** *After product formation, row i contains the $(m+1)$-bit representation of Ab_i in columns $-i-1, \ldots, m-i-1$, with a leading 0. In global column weights, row i represents $Ab_i 2^i$. All remaining cells contain blanks.*

Proof. The expert forms products from row $n-1$ upwards. On row i, the expert erases the rightmost factor bit b_i, the remaining factor prefix, and x, then walks left across a. For $b_i = 0$, the expert writes zeros; for $b_i = 1$, the expert preserves the digits of a. Both branches write one additional 0 on the left blank. The resulting word has the claimed support and value, with a leading zero because $Ab_i < 2^m$.

The expert scans right to the blank at column $m-i$ and moves up. For $i > 0$, the upper cell contains the last digit of a on row $i-1$; the next cell contains x. The expert scans the remaining factor prefix, selects the last bit, and repeats. Each upper prefix contains at least two bits, so another digit follows the selected bit during leftward erasure. For $i = 0$, the upper cell contains a blank, ending product formation and starting summation on row 0. □

Both passes terminate: each copy shortens the remaining factor prefix, and product formation visits each of the n rows once. Every horizontal scan crosses a finite word.

2.2 A Fixed-Width Addition Invariant

Lemma 3 (Accumulated Product)**.** *After merging through row k, $0 \le k < n$, all rows above k contain only blanks. Row k contains a contiguous $(m + k + 1)$-bit accumulator on columns $-k - 1, \ldots, m - 1$, with value*

$$S_k = A \sum_{i=0}^{k} b_i 2^i.$$

Rows below k retain the partial products from Lemma 2. At the inspection of the next row, the expert occupies state 20 with head $(-k - 2, k + 1)$.

Proof. For $k = 0$, the initial left scan crosses the product on row 0, reads the leading 0, reaches the blank at column -2, and descends.

If row $k + 1$ exists, the inspected cell contains the leading 0 at column $-k - 2$. The expert scans right across the lower product to the first blank, at column $m - k - 1$. Moving up reaches an accumulator digit because

$$-k - 1 \;\le\; m - k - 1 \;\le\; m - 1.$$

The expert scans the remaining upper digits, reaches the right blank, and steps left to column $m - 1$. The lower cell contains a blank because the original lower product ends at $m - k - 2$. The first column therefore adds the upper bit to zero without a carry.

The expert processes the remaining columns from right to left. For upper bit u, lower bit v (with a blank counting as zero), and incoming carry c, the update satisfies

$$u + v + c = w + 2c', \qquad w \in \{0, 1\}, \quad c' \in \{0, 1\}. \tag{2}$$

The expert erases the upper bit, writes w below, and stores c' in the control state. The processed lower suffix contains the correct sum suffix, the unprocessed cells retain the previous values, and the carry records the contribution to the next column. Equation (2) preserves this invariant.

The original lower word extends one column farther left than the upper word, with a leading 0 at $-k - 2$. After consuming the upper word, carry zero preserves this bit, whereas carry one changes the bit to 1 and clears. The expert reaches the blank at $-k - 3$ on row $k + 1$ and descends. Row k now contains only blanks, and row $k + 1$ represents $S_k + Ab_{k+1}2^{k+1} = S_{k+1}$ on the claimed support. □

Lemma 4 (Carry Confinement)**.** *During an upper-digit update with incoming carry 1, the lower cell contains an original binary digit. If the carry outlives the upper word, the next lower cell contains the original leading 0.*

Proof. A blank gap lies to the right of the original lower product. The first column has carry zero, and $u + 0 + 0 \le 1$, so no carry originates in the gap. A carry first arises inside the lower digit block. Further leftward steps remain inside that block until the leading zero. The upper word ends one column before that zero, where $0 + 0 + 1 = 1$ clears the carry. □

Each merge consumes finitely many upper digits and at most one extra carry column. After $n - 1$ merges, state 20 reads the blank below the final row and moves up to state 30, which emits the final S action and enters halt state 31. Lemma 3 gives the $(m + n)$-bit product and final head $(-n - 1, n - 1)$. All other cells contain blanks.

3 Observation Protocol

The observation protocol refines expert states by geometric context. Each phase records the permitted read symbols and successor phases. Section 4 gives the complete protocol together with the controller-state relation; the shared tables avoid repeating the same phase definitions.

Lemma 5 (Protocol Coverage)**.** *Every expert execution on a well-formed multiplication follows the* 67 *phases in Section 4, from phase* 0 *to phase* 66*.*

Proof. Lemma 1 supplies the copy-phase conditions; Lemma 2 supplies the product-phase conditions. Lemmas 3 and 4 supply the alignment, addition, carry and boundary conditions. The explanations below the three tables account for the first and last cell readings in each phase. Induction over the individual transitions, the copied rows, and the merged rows establishes coverage. The finite-pass arguments in the expert invariants give termination in the final phase. □

4 Simulation Certificate

4.1 The Copying Phases

The following three tables define a finite protocol and a simulation relation. Phase i pairs expert state g_i with controller state $q \in Q_i$ on an identical grid and head. Each set Q_i contains the single listed state. An entry

$$x : (j, w, M)$$

means: read x, write w, move M, and advance to phase j. Here $d \in \{0, 1\}$ denotes either digit, consistently within each entry. The index j denotes a phase. Successors comprise expert state g_j and controller state $q' \in Q_j$. The certificate checks these transitions against both exports. The protocol starts at phase 0.

i	Phase	g_i	Q_i	Read : (next phase, write, move)
0	First left-operand digit	0	$\{0\}$	$d : (1, d, \mathsf{R})$
1	Scan left operand	0	$\{1\}$	$d : (1, d, \mathsf{R})$ $x : (2, x, \mathsf{R})$
2	First right-operand digit	0	$\{2\}$	$d : (3, d, \mathsf{R})$
3	Scan right operand	0	$\{3\}$	$d : (3, d, \mathsf{R})$ $\lambda : (4, \lambda, \mathsf{L})$
4	Skip last right-operand digit	1	$\{4\}$	$d : (5, d, \mathsf{L})$
5	Copy more, or start products	2	$\{5\}$	$0 : (6, 0, \mathsf{D})$ $1 : (8, 1, \mathsf{D})$ $x : (22, x, \mathsf{R})$
6	First copied 0: descend	3	$\{52\}$	$\lambda : (7, \lambda, \mathsf{L})$
7	First copied 0: write	6	$\{53\}$	$\lambda : (10, 0, \mathsf{U})$
8	First copied 1: descend	4	$\{6\}$	$\lambda : (9, \lambda, \mathsf{L})$
9	First copied 1: write	7	$\{7\}$	$\lambda : (10, 1, \mathsf{U})$
10	Continue after first copy	9	$\{8\}$	$0 : (11, 0, \mathsf{D})$ $1 : (13, 1, \mathsf{D})$ $x : (16, x, \mathsf{D})$
11	Copy 0 above a lower digit	3	$\{16\}$	$d : (12, d, \mathsf{L})$
12	Write copied 0	6	$\{17\}$	$\lambda : (15, 0, \mathsf{U})$
13	Copy 1 above a lower digit	4	$\{9\}$	$d : (14, d, \mathsf{L})$
14	Write copied 1	7	$\{10\}$	$\lambda : (15, 1, \mathsf{U})$
15	Continue the copy	9	$\{11\}$	$0 : (11, 0, \mathsf{D})$ $1 : (13, 1, \mathsf{D})$ $x : (16, x, \mathsf{D})$ $\lambda : (21, \lambda, \mathsf{D})$
16	Descend below operator	5	$\{12\}$	$d : (17, d, \mathsf{L})$
17	Write shifted operator	8	$\{13\}$	$\lambda : (18, x, \mathsf{U})$
18	Copy last left-operand digit	9	$\{14\}$	$0 : (19, 0, \mathsf{D})$ $1 : (20, 1, \mathsf{D})$
19	Copied 0 above lower operator	3	$\{55\}$	$x : (12, x, \mathsf{L})$
20	Copied 1 above lower operator	4	$\{15\}$	$x : (14, x, \mathsf{L})$
21	Begin scan on copied row	0	$\{0\}$	$d : (1, d, \mathsf{R})$

Why the Listed Read Symbols Cover Every Copy. The nonempty operands ensure that the initial scan reads a digit, the separator, and at least one factor digit before reaching a blank. The second left step sees either a digit to copy or x when only one factor bit remains. Lemma 1 identifies the three possible cells below a copied digit: blank for the first copy, x immediately after copying the separator, and a digit otherwise. Every destination contains a blank. Copying the first digit of a exposes the left blank; restarting on the new row therefore reads a digit. These facts establish the table's geometric restrictions.

The protocol permits arbitrary operand lengths and may admit additional observation sequences. Every valid execution follows permitted branches; checking all such branches therefore establishes the required local agreement.

4.2 The One-Bit Product Phases

Phase i pairs expert state g_i with controller state $q \in Q_i$. An edge $x : (j, w, M)$ reads x, writes w, performs motion M, and advances to phase j. The product phases preserve the exact leading-zero convention of the expert.

i	Phase	g_i	Q_i	Read : (next phase, write, move)	
22	Read the last remaining factor bit	10	{18}	$0 : (23, \lambda, \mathsf{L})$	$1 : (26, \lambda, \mathsf{L})$
23	Erase operator for factor 0	11	{56}	$x : (24, \lambda, \mathsf{L})$	
24	First digit of product by 0	13	{32}	$d : (25, 0, \mathsf{L})$	
25	Continue product by 0	13	{33}	$d : (25, 0, \mathsf{L})$	$\lambda : (29, 0, \mathsf{R})$
26	Erase operator for factor 1	12	{19}	$x : (27, \lambda, \mathsf{L})$	
27	First digit of product by 1	14	{20}	$d : (28, d, \mathsf{L})$	
28	Continue product by 1	14	{21}	$d : (28, d, \mathsf{L})$	$\lambda : (29, 0, \mathsf{R})$
29	Scan completed partial product	15	{22}	$d : (29, d, \mathsf{R})$	$\lambda : (30, \lambda, \mathsf{U})$
30	Inspect the row above	16	{23}	$d : (31, d, \mathsf{R})$	$\lambda : (39, \lambda, \mathsf{D})$
31	Cross upper operator	17	{24}	$x : (32, x, \mathsf{R})$	
32	First bit in upper factor prefix	17	{25}	$d : (33, d, \mathsf{R})$	
33	Scan upper factor prefix	17	{26}	$d : (33, d, \mathsf{R})$	$\lambda : (34, \lambda, \mathsf{L})$
34	Read upper row factor bit	10	{27}	$0 : (35, \lambda, \mathsf{L})$	$1 : (37, \lambda, \mathsf{L})$
35	First remaining bit to erase (0)	11	{30}	$d : (36, \lambda, \mathsf{L})$	
36	Erase factor prefix (0)	11	{31}	$d : (36, \lambda, \mathsf{L})$	$x : (24, \lambda, \mathsf{L})$
37	First remaining bit to erase (1)	12	{28}	$d : (38, \lambda, \mathsf{L})$	
38	Erase factor prefix (1)	12	{29}	$d : (38, \lambda, \mathsf{L})$	$x : (27, \lambda, \mathsf{L})$

First and Subsequent Product Rows. The bottom row contains one factor bit. Erasing that bit moves the head directly onto x. Every higher row has at least two right-operand bits: after erasing the selected last bit, the expert erases another digit before reaching the separator. These facts justify the separate "single" and "first remaining bit" phases in the table.

Upper-Row Alignment. A finished product occupies $[-i - 1, m - i - 1]$ on row i. The right scan ends at column $m - i$. An existing upper row has the last left-operand digit at that column, followed immediately by x. After that separator comes a nonempty remaining right operand. If no row above exists, the expert reads a blank and starts summation. Lemma 2 proves these conditions for every operand length, including operands containing only zeros.

Product-Scan Coverage. Multiplication by 0 turns each left-operand digit into 0; multiplication by 1 preserves each digit. Both passes have at least one digit to process and finish on a blank by writing a padding 0. The subsequent right scan sees only digits followed by a blank. A completed product row contains no separators.

4.3 Summation, Carry and the Final Stop

These phases refine the expert's addition states by the position in a merge and the carry condition. This refinement supplies every required transition in the partial controller table on all well-formed multiplications.

i	Phase	g_i	Q_i	Read : (next phase, write, move)		
39	Start summing the product rows	18	{34}	$\lambda : (40, \lambda, \mathsf{L})$		
40	First digit of initial left scan	19	{35}	$0 : (41, 0, \mathsf{L})$	$1 : (42, 1, \mathsf{L})$	
41	Left scan after reading zero	19	{36}	$0 : (41, 0, \mathsf{L})$	$1 : (42, 1, \mathsf{L})$	$\lambda : (43, \lambda, \mathsf{D})$
42	Left scan after reading one	19	{35}	$0 : (41, 0, \mathsf{L})$	$1 : (42, 1, \mathsf{L})$	
43	Inspect fresh lower leading cell	20	{37}	$0 : (44, 0, \mathsf{R})$	$\lambda : (65, \lambda, \mathsf{U})$	
44	First digit after lower leading zero	21	{22}	$d : (45, d, \mathsf{R})$		
45	Scan the lower partial product	21	{22}	$d : (45, d, \mathsf{R})$	$\lambda : (46, \lambda, \mathsf{U})$	
46	First upper alignment digit	28	{23}	$d : (47, d, \mathsf{R})$		
47	Next upper alignment cell	28	{24}	$d : (48, d, \mathsf{R})$	$\lambda : (49, \lambda, \mathsf{L})$	
48	Finish aligning the upper row	28	{45}	$d : (48, d, \mathsf{R})$	$\lambda : (49, \lambda, \mathsf{L})$	
49	Read upper units digit	22	{38}	$0 : (50, \lambda, \mathsf{D})$	$1 : (51, \lambda, \mathsf{D})$	
50	Write units digit 0 below	24	{39}	$\lambda : (52, 0, \mathsf{U})$		
51	Write units digit 1 below	25	{54}	$\lambda : (52, 1, \mathsf{U})$		
52	Return above, carry zero	23	{40}	$\lambda : (53, \lambda, \mathsf{L})$		
53	Read upper bit, carry zero	22	{41}	$0 : (54, \lambda, \mathsf{D})$	$1 : (55, \lambda, \mathsf{D})$	$\lambda : (60, \lambda, \mathsf{D})$
54	Add value zero below	24	{42}	$d : (52, d, \mathsf{U})$	$\lambda : (52, 0, \mathsf{U})$	
55	Add value one below	25	{46}	$\lambda : (52, 1, \mathsf{U})$	$0 : (52, 1, \mathsf{U})$	$1 : (56, 0, \mathsf{U})$
56	Return above, carry one	27	{47}	$\lambda : (57, \lambda, \mathsf{L})$		
57	Read upper bit, carry one	26	{48}	$0 : (58, \lambda, \mathsf{D})$	$1 : (59, \lambda, \mathsf{D})$	$\lambda : (62, \lambda, \mathsf{D})$
58	Add one inside lower digit block	25	{50}	$0 : (52, 1, \mathsf{U})$	$1 : (56, 0, \mathsf{U})$	
59	Add two inside lower digit block	29	{49}	$d : (56, d, \mathsf{U})$		
60	Preserve unneeded leading zero	19	{43}	$0 : (61, 0, \mathsf{L})$		
61	Blank to the left of merged row	19	{44}	$\lambda : (43, \lambda, \mathsf{D})$		
62	Apply final carry to leading zero	25	{57}	$0 : (63, 1, \mathsf{U})$		
63	Return after final carry	23	{58}	$\lambda : (64, \lambda, \mathsf{L})$		
64	Step down left of merged row	22	{59}	$\lambda : (61, \lambda, \mathsf{D})$		
65	Emit final stop	30	{51}	$\lambda : (66, \lambda, \mathsf{S})$		
66	Halted	31	{60}	–		

Boundary Restrictions. The initial left scan can encounter a blank only after reading the product's leading 0. The next inspection reads either a blank or the fresh row's leading 0. Alignment always starts on an upper digit, and the first lower units cell contains a blank. Lemma 4 excludes lower blanks in the carry-one digit phases and forces 0 for the final carry extension. With no final carry, the expert preserves the untouched leading 0; after either branch, the next cell to the left of the result contains a blank. The final stop also reads a blank.

These facts, together with contiguity of every accumulator, justify all read restrictions in this table. The restrictions exclude every missing transition that could otherwise obstruct a valid multiplication.

Local Transition Identity. For each of the 130 listed read/action cases and the unique $q \in Q_i$, direct comparison with the two exported tables gives

$$\delta_{\mathcal{E}}(g_i, x) = (g_j, w, M), \qquad \delta_C(q, x) = (q', w, M), \qquad q' \in Q_j. \tag{3}$$

Every required entry exists. A transition reaches a halt state exactly when $j = 66$, with motion S. The certificate covers all 119 defined nonterminal controller entries. The remaining four entries contain halt loops.

5 Equivalence Proof

Proof of Theorem 1. Initially both configurations have the same grid and head, with control states 0 related by phase 0. Suppose the relation holds before a controller step. Lemma 5 supplies an allowed read symbol x. Equal grids and heads give the same read symbol to both controllers. Equation (3) gives equal writes and motions and preserves the relation between successor states. The next grids and heads therefore agree.

Induction establishes equality of every finite execution prefix. The terminal condition permits a halt exactly on an edge to the terminal phase. The finite copy, product, and merge passes establish termination. Both controllers therefore halt together with equal complete PTOs and final configurations. Lemma 3 gives the product, padding, and final head. □

6 Verification and Artifacts

Verification. Run `python3 verify_equivalence.py --quick` from this proof directory to check the file fingerprints, CSV/JSON agreement, equality of the exported expert with `mult_grid(2)`, and all 130 certificate obligations. Omitting `--quick` also compares independent executions on all 15,876 operand pairs of lengths 1–6, plus 45 directed and 128 fixed-seed random cases. All 10,184,741 transitions agree, including cases with 1024-bit operands. The execution checks exercise every defined nonterminal controller entry and compare final grids, heads, and the padded arithmetic product. The invariants and induction establish the unbounded-length claim.

Artifact Identity. The title page gives the repository-relative case directory. This proof directory contains the controller and expert exports, reference code, protocol, checker, and saved verification report. The verification script stores or reads SHA-256 checksums for these artifacts. A checksum identifies the exact bytes of a file; the checker compares the stored and computed values to detect changes. These checks bind the certificate to the supplied exports. The invariants, protocol coverage, and local transition identity establish equivalence.